

Legacy Lift



Legacy Lift



Legacy Lift

Yesterday's Problem → Tomorrow's Solution

Table of Contents



- Meet the Team
- Executive Summary
- High Level Overview
- Development Tools
- Deliverables
- Work Breakdown Structure
- Comparisons
 - Code Comparison - Cobol
 - Code Comparison - Python
 - Infrastructure Comparison
 - RWP and Legacy Lift
- Current Process Flow
- Proposed Solution Flow
- Solution Statement
- Features
- Limitations
- Competition Matrix
- Risks
 - Legal Risk and impacts
 - Risk Analysis and mitigation
- Major functional components
 - COTS and legacy software
- Agile Development Model
- Algorithms
- Database Schema
- Monetization
- Libraries and Frameworks
- RAG
 - What it is
 - How it works
 - Why it matters
- Major Functional Components Diagram
- Data Flow Diagram
- Project GUI Mockup
- Conclusion
- References
- Appendix - User Stories

Meet the Team

Spencer Peloquin - He is enrolled in the BSCS MS program at Old Dominion University. He has completed a technical internship at Tacoma Public Utilities. In addition, he has experience in Python, Java, Ruby, C++, C#, SQL and Fortran. He lives in Tacoma WA, where he enjoys running, reading and hiking.



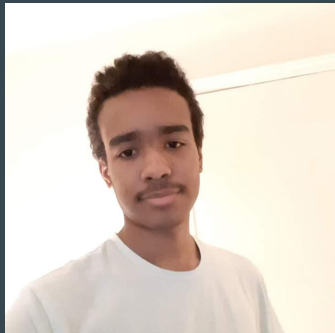
Aaditya Ghimire(AG)- He is enrolled in Computer Science Program at ODU. He has experience in Java + Spring Boot and with main focus on Python deep learning computer vision.



Caleb Peterson - He is enrolled in the bachelors computer science program at ODU. He has experience in Java, JavaScript, C++, and SQL. His hobbies include video games and board games.



Darius Itam - He is enrolled in the bachelors computer science program as a senior at ODU main campus in Norfolk, VA. He is proficient in Python, Java, JavaScript, and C#. He loves fitness, esports, and music.



Justin Tendilla - CS Major. Loves playing guitar, video games, and movies. Experienced in C++ and Java.

Jordan Kemelhar - IT/Azure cloud professional and Senior CS student at Old Dominion University. Certified in CyberSecurity Has experience in Java and Azure user management. Hobbies video games, and, hiking, board games.



Executive Summary

AI Pipeline for Legacy Code Modernization



Problem

Many critical enterprise systems still run on legacy software written in outdated languages such as COBOL. These systems are deeply embedded in industries such as finance, government, and enterprise infrastructure. 95% of ATM transactions in the U.S. are processed using COBOL-based systems, and over 60% of U.S. hospitals run critical applications on legacy software such as COBOL and Fortran (Pragmatic Coders, 1).

Impact

The problem is that legacy systems introduce security risks, compatibility issues with modern technology, and large amounts of technical debt. Modernizing them is extremely difficult because these systems contain complex dependencies and decades of accumulated business logic. It is especially important to port code to modern platforms for use in cloud applications, which are primarily incompatible with legacy systems.

Solution

Legacy Lift proposes an AI-driven pipeline that analyzes dependencies and incrementally converts legacy systems to modern infrastructure. It will maintain full compatibility with the infrastructure of the legacy code while enabling the security/performance benefits of modern programming languages.



High Level Overview

Purpose:

- Translate legacy code from COBOL to modern Python 3.11
- Solve technical debt with legacy codebases
- Resolve ongoing security and memory issues with legacy systems

Mechanics:

- Uses Gemini AI to convert code
- Translates legacy plugin dependencies into modern package hooks
- Preserves class structure from original code

Benefits:

- Functional equivalent code conversion
- Leverages both Generative AI and human review
- Provides an on-ramp to modern systems

COBOL - Background

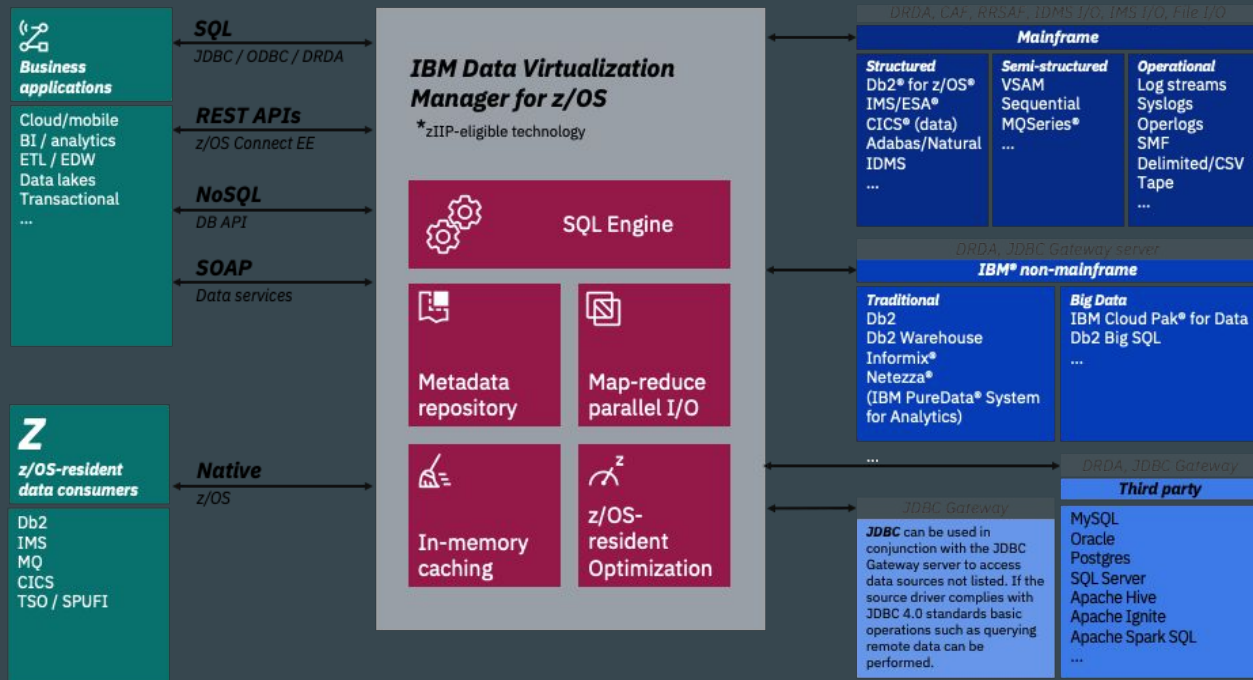
COBOL is a high-level programming language designed specifically for business data processing.

- Key characteristics:
 - English-like syntax (e.g., ADD SALES-AMOUNT TO TOTAL-SALES).
 - Extremely strong in file handling, report generation, and decimal arithmetic (critical for financial calculations to avoid rounding errors).
 - Compiled, procedural, and very verbose.
- History
 - Developed in 1959 by Grace Hopper.
 - Still one of the most widely used languages in production for core banking and insurance systems.
- Modern status
 - Millions of lines of COBOL still run daily on mainframes.
 - Many organizations are gradually modernizing but wholesale replacement is rare due to risk and cost.

IBM z/OS

- IBM's flagship 64-bit operating system for IBM Z mainframes (formerly System/360 lineage).
- Purpose-built for mission-critical, high-volume enterprise workloads in banking, insurance, government, healthcare, and retail
- Usage: z/OS is the native platform COBOL is designed for
- Stability: z/OS is designed to be run 24/7 without interruption, limiting maintenance
- Scalability: Handles thousands of concurrent users and enormous I/O and batch processing. It excels at mixed workloads (online transactions + large batch jobs).
- Backward Compatibility: Retains functionality from the 1960s OS/360 era. Old applications often run with little or no change

z/OS Mainframe Architecture LegacyLift



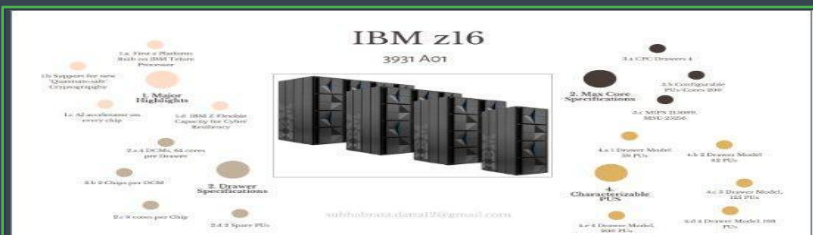
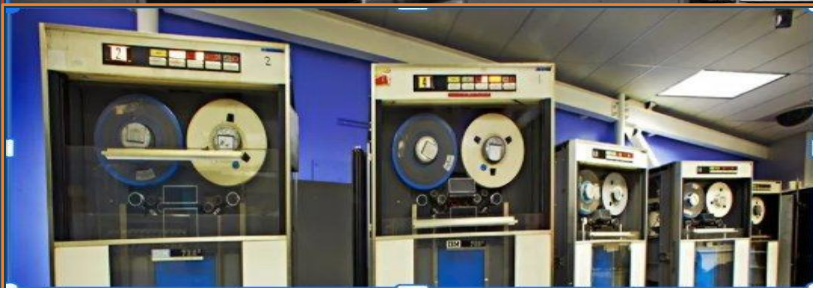
Access to unlisted data sources is not optimized and may not support certain data types, functions or SQL operations.

The Legacy Hardware

COBOL ran on all three — the language outlived the machines



Before — 1960s



60s

IBM System/360

Engineers at the IBM System/360 operator room. Crew-cut engineers, magnetic tape reels, punch-card input. This is where enterprise computing — and COBOL — was born.

70s

Storage — 1970s

IBM magnetic tape drives — each reel held ~45 MB. No cloud. No instant access. If you needed data, someone physically mounted a reel.

Magnetic tape drives

2022

IBM z16

200 configurable cores, quantum-safe cryptography, AI acceleration on every chip. Same COBOL code. The hardware evolved dramatically. The software didn't.

95% of U.S. ATM transactions processed via COBOL **60%+** of U.S. hospitals run critical systems on legacy code **That's the problem Legacy Lift solves.**

Code Comparison- Cobol to Python

```
identification division.
program-id. is-numeric-test.
data division.
working-storage section.
01 ws-user-input pic x(10).
01 ws-user-input-justified pic x(10) justified right.
procedure division.
main-procedure.
    perform process-plain
    perform process-zero-fill
    perform process-trim
    stop run.
process-plain.
    display "(plain) Enter a value: " with no advancing
    accept ws-user-input
    if ws-user-input is numeric then
        display ws-user-input " is numeric!"
    else
        display ws-user-input " is not numeric."
    end-if
    exit paragraph.
process-zero-fill.
    display "(right justify, zero fill) Enter another value: "
    with no advancing
    accept ws-user-input-justified
    inspect ws-user-input-justified
        replacing leading spaces by '0'
    if ws-user-input-justified is numeric then
        display ws-user-input-justified " is numeric!"
    else
        display ws-user-input-justified " is not numeric."
    end-if
    exit paragraph.
process-trim.
    display "(trim) Enter a third value: " with no advancing
    accept ws-user-input
    if function trim(ws-user-input) is numeric then
        display function trim(ws-user-input) " is numeric!"
    else
        display function trim(ws-user-input) " is not
numeend-if
    exit paragraph.
end program is-numeric-test.
return go(f, seed, [])
}
```

50
COBOL LINES

15

Python Lines
70% Reduction

```
def is_numeric(value):
    try:
        float(value)
        return True
    except (ValueError, TypeError):
        return False

user_input = input("Enter something: ").strip()
if is_numeric(user_input):
    print(f"'{user_input}' is numeric!")
    if user_input.isdigit():
        print(" It's an integer.")
    else:
        print(" It's a floating-point
else:
    print(f"'{user_input}' is NOT numeric.")
```

Job Control Language (JCL)

What it is

The scripting language that tells a mainframe OS what to run. Almost everything in batch on z/OS goes through JCL — including every COBOL job.

Key Characteristics

- Not a general-purpose language — it's control language for batch jobs
- EXEC statements define each program step to run
- DD statements assign input/output files and datasets
- Defines resources, parameters, and job accounting
- Scheduling systems (Tivoli, Control-M) submit JCL streams

Unix — copy a file

```
cp oldFile newFile
```

vs.

JCL — same operation on z/OS

```
//MYJOB    JOB    (ACCT), 'COPY FILE'  
//STEP1    EXEC   PGM=IEBGENER  
//SYSPRINT DD   SYSOUT=*  
//SYSIN     DD   DUMMY  
//INPUT     DD   DSN=MY.DATA, DISP=SHR  
//OUTPUT    DD   DSN=RESULTS.FILE, ...
```



Prototype Scope: LegacyLift converts COBOL source code only. JCL orchestration is **not addressed** in this prototype.

Fred Brooks (OS/360 architect) called JCL “the worst computer programming language ever devised by anybody, anywhere.”

Work Breakdown Structure

LegacyLift Full Development Breakdown

Everything to Build, Set Up, Design, or Test — AI-Assisted Fortran & COBOL Analysis Tool for Python Modernization



Deliverables

Updated Python code with functional equivalence to the legacy code.

Changes documentation with a collection of log files and original COBOL functions.

RAG justification for python changes.

System tests to test for compatibility

Vector Database for Retrieval Augmented Generation

LegacyLift code update toolchain

Deployable self contained LegacyLift Docker environment for demo

Development Tools



- **Frontend**

- Python TK based widget
- Electron wrapper



- **Backend**

- Python 3.11
- Gemini



- **Database / Storage**

- MySQL Server
- Vector Database for RAG retrieval



- **Development / IDE**

- Visual Studio Code



- **Version Control**

- GitHub + Git



- **Environment / Infrastructure**

- Windows 11/Ubuntu 24.04
- Miniconda
- Docker



Infrastructure Comparison

Legacy COBOL environment vs. Modern Python — why the gap matters



Feature	Legacy Code (COBOL/Ada)	Modern Python Environment
Language	COBOL, Fortran, Ada	Python 3.11+
Developers	Scarce — shrinking talent pool	Millions: most taught globally
IDE / Tooling	Proprietary, limited debuggers	VSCode, linters, extensions
Testing	Manual, minimal frameworks	PyTest, Automated CI pipelines
Documentation	Often missing or non-standard	Docstrings, auto-generated docs
Version Control	None or proprietary systems	Git + GitHub + GitHub Actions
Security	Slow patches, legacy vulnerabilities	Active community, rapid patching
Maintenance Cost	High: Specialist rates (\$150+/hr)	Standard: Competitive market rates

Legacy limitations

Modern advantages

Legacy Lift bridges this gap.

Comparisons

Infrastructure: Legacy vs Modern		
Feature	Legacy (COBOL/Ada)	Modern Python
Language	COBOL, Fortran, Ada	Python 3.11+
Developers	Scarce	Millions globally
IDE / Tooling	Proprietary, limited debuggers	VSCode, linters, extensions
Testing	Manual, minimal frameworks	PyTest, Automated CI
Documentation	Often missing or non-standard	Docstrings, Auto-generated
Version Control	None or proprietary systems	Git + GitHub Actions
Security	Slow patches, legacy vulns	Active community, rapid patching
Maintenance Cost	High: \$150+/hr specialist rates	Standard: Competitive market

Feature Matrix: RWP vs Legacy Lift		
Feature	RWP	Legacy Lift
COBOL → Python 3.11 Translation	✓	✓
Fortran → Python Translation	✓	—
Ada → Python Translation	✓	—
Recreate Original Class Structure	✓	—
Modern Design Refactoring	✓	✓
Runs on Modern x86 Systems	✓	✓
Plugin / Framework Compatibility	✓	—
Preserve Comments & Documentation	✓	✓
Audit Logging	✓	—
Functional Equivalence	✓	✓
Dependency Analysis	✓	✓
Multi-file / Large Codebase	✓	—
CI/CD Integration (Enterprise)	✓	✓
Security Vulnerability Scanning	✓	—

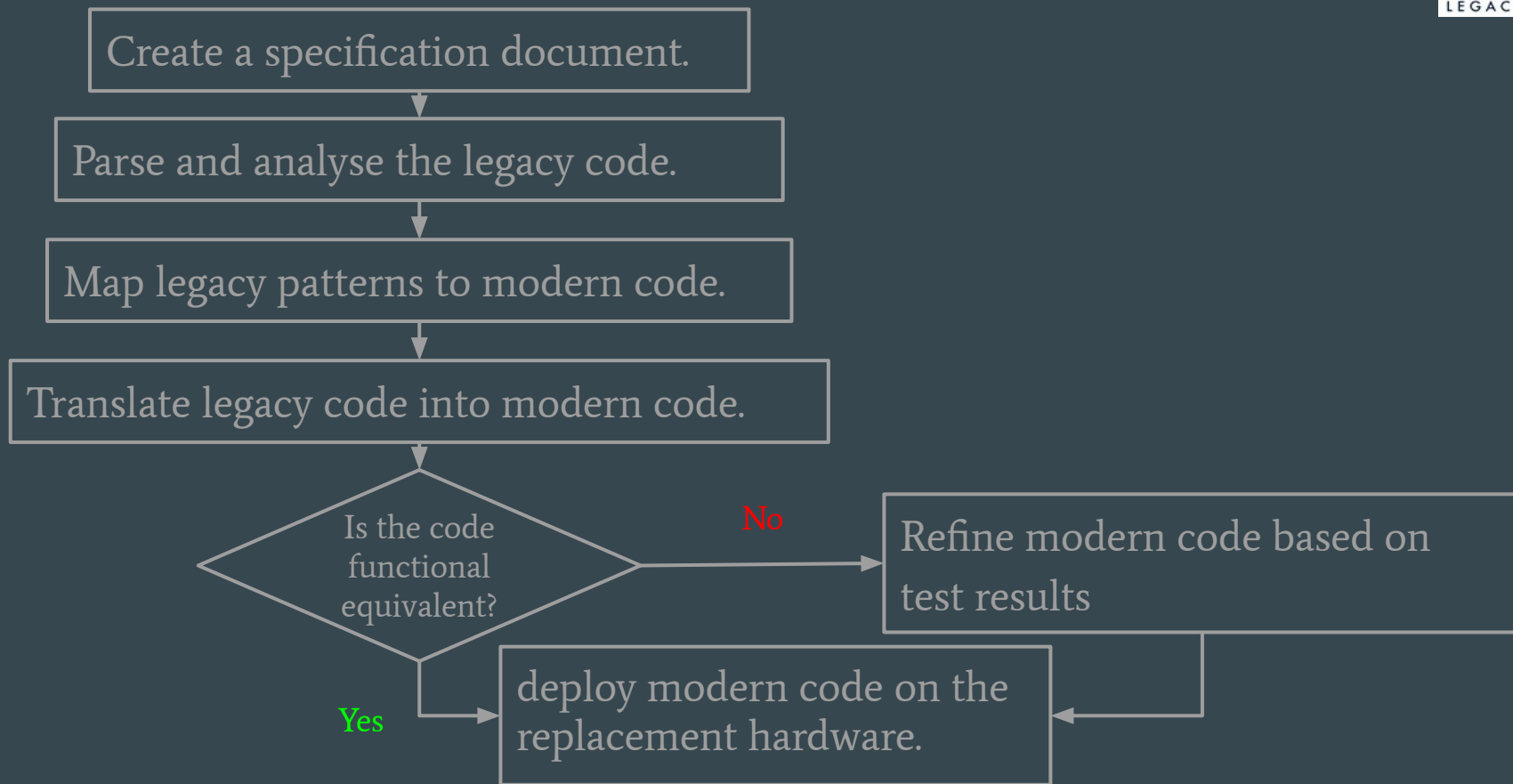
Legacy limitations

Modern advantages

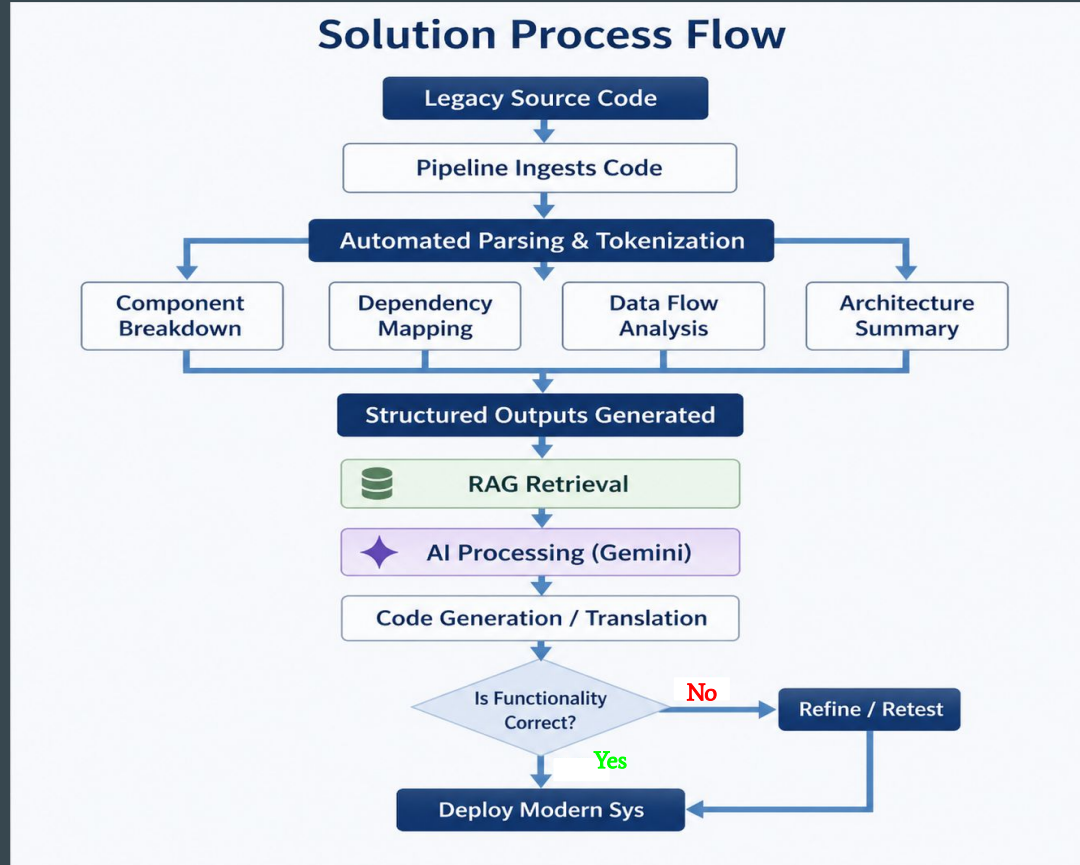
✓ = Supported

— = Not available

Current Process Flow



Solution Process Flow





Solution Statement

This project will produce a prototype software pipeline that analyzes legacy source code and generates structured outputs describing the code's components and relationships, demonstrating the feasibility of using AI-assisted techniques to support legacy system modernization.

Software Type

- AI-assisted legacy code analysis tool

Purpose

- AI-assisted analysis of legacy source code
- Marketed for Consultants.

High-level Function

- Accept legacy code as input
- Analyze structure and dependencies
- Produce structured outputs describing the system



Features

Code Translation

- COBOL -> Python Conversion
- Preserve core logic and program structure

System Analysis

- Identify dependencies and relationships
- Generate structured system representations

Data Awareness

- Detect data access (files, ISAM, DB2)
- Extract record structures and I/O operations

Modernization & Refactoring

- Transform legacy code into modern Python constructs
- Support object-oriented representations where applicable
- Enable execution on modern systems

Reliability & Support

- Maintain documentation and comments
- Provide audit logs of translation steps
- Support human validation and refinement

Limitations

- Be able to run unmodified on legacy hardware
- Run its own code
- Does not migrate databases (e.g., DB2)
- Does not convert legacy file systems (ISAM, flat files)
- Translate or decompile binary modules
- Retain any or all technical debt
- Assume missing functionality
- Access the network or external/cloud drives
- Guarantee the absence of bugs or security issues
- Convert legacy code into code of a completely different nature
- Function as a chatbot, game engine or “swiss army knife tool”

Competition Matrix



FEATURES	Our Product	ChatGPT	IBM Watsonx Code Assistant	Azure Migrate
Legacy Code Conversion	✓	✓	✓	✓
Dependency Analysis	✓			✓
Full Compatibility with Plugins or Frameworks	✓			
AI Incremental Translation Pipeline	✓			
New Code has Functional Equivalence with old code	✓		✓	✓
Audit log of changes	✓			✓
Modern OS/x86 Compatibility	✓		✓	✓

Competition Analysis

- Current competition includes IBM and Microsoft.
- These models use in house AI conversion tools based on Copilot models.
- They also come with enterprise support and are open source.
- The key difference between our model (Legacy Lift) and theirs is that our model uses the Gemini model.
- It is fully backed by a University CS department and does not collect data for sale.
- Other research projects from universities like Stanford and MIT hold this model, and should provide proper benefits for a company.

Legal Risks

- **Primary Risk** : Users over-trust AI in the COBOL-to-Python translator, assuming consistent accuracy despite frequent hallucinations in code logic, data handling, or refactoring.
- **Real-World Evidence** : Numerous lawsuits and sanctions against AI users (e.g., lawyers fined \$1,500–\$86,000 for hallucinated legal citations); enterprise hallucination losses estimated at **\$67.4 billion** in 2024.
- **Probability** : High — AI confidently wrong outputs are common (e.g., 17–34% hallucination rates even in specialized legal AI tools).
- **Impact Rating** : 4/5 — Heavy financial (fines, rework, lost revenue) and reputational costs for a small modernization startup.
- **Specific Relevance** : Critical for legacy migrations where errors in translated COBOL business rules or Fortran numerical code can cause production failures or compliance issues.
- **Key Recommendation** : Strong disclaimers, mandatory human review, and clear limitation disclosures in all client engagements to reduce liability.

Risk Analysis

- Incompatibility & Technical Debt
- Compatibility between legacy and modern Python
- Gemini produces hallucinated Python logic
- Migration Failure Risks
- AI Over-Trust Amplification
- Deployed code behaves differently in production

Mitigations:

- Disclaimers
- human review
- Audit trail of model version
- Runtime compatibility
- Full audit logging and documentation



Major Functional Components

Tech stack:

L - Container - Docker

A - Cloud - Google GCP

M - DB - MySQL

P - Language - Python

Python CLI -> Provides the Conda shell to run the code

Program runs -> The Legacy Lift Software is initialized on the cloud PC

Code Parsing -> The Legacy Lift Software takes tokens and classes and sends them to Gemini

Sends to Gemini -> Gemini parses these tokens and generates compatible classes

Dependency analysis -> The tokens are analyzed for possible dependencies and compatible classes

Converts Code to target language -> These classes are rebuilt for Python with a focus of creating modern code over preserving legacy quirks

Verify integrity -> check for corruption and verify all classes are valid

Makes sure it can run -> run the first 10,000 tokens on the bare metal for stress testing

Validates one to one functionality -> A series of unit, system and integration tests are run to ensure 1:1 compatibility

Deploys new code -> create a Conda Environment to deploy new code

COTS and Legacy Systems

COTS:

LegacyLift will use Gemini as the primary translation backbone. Retrieval Augmented Generation will be accomplished by using Gemini in a vector database.

MySQL will be the primary database implementation.

Uses GCP with docker so LegacyLift can be run anywhere with any code size

Legacy Systems:

Uses GnuCOBOL as an interpreter for COBOL

Agile Development Model

Agile development method

Chosen because LegacyLift works like a startup

Organized using a scrum

Waterfall and RUP are not suitable because LegacyLift is not enterprise

Algorithms

Parsing & Abstract Syntax Tree (AST) Construction

Component Detection & Inventory

Dependency & Relationship Analysis

AI-Assisted Analysis Layer

Graph & Visualization Algorithms

Supporting / Utility Algorithms

Libraries and Frameworks

- FastAPI - API Framework
- Psycopg2 - PostgreSQL driver
- ANTLR4 Python runtime - COBOL/Fortran Parsing
- NetworkX - Network Graphing
- LangChain - Gemini Integration
- Python-dotenv - Environment Management
- React - Frontend
- Pytest - Unit testing
- f2py / numpy / scipy - Data Manipulation

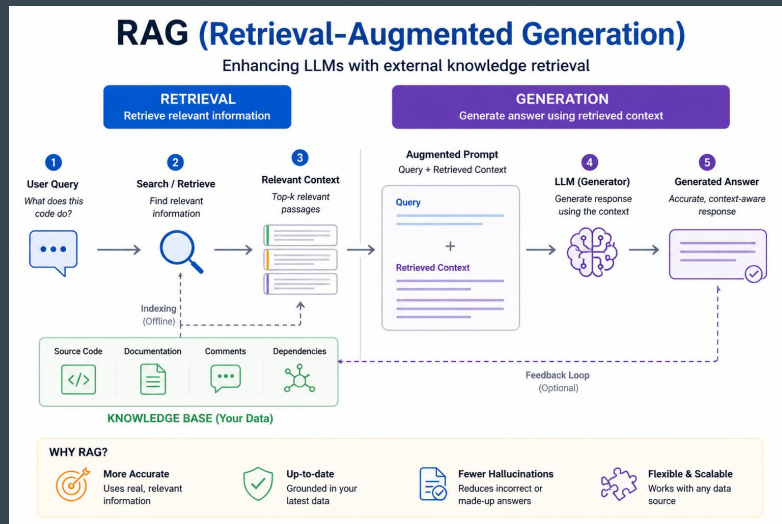
RAG - What It Is

Retrieval-Augmented Generation

- Combines data retrieval + AI generation
- Retrieves relevant code, documentation, and dependencies before AI processing
- Provides context to the model instead of relying on memory alone

RAG - How It Works

- Legacy code and documentation are split into searchable chunks
- Relevant context is retrieved based on the current task
- Retrieved data is added to the prompt
- Gemini generates analysis or translated code utilizing that context



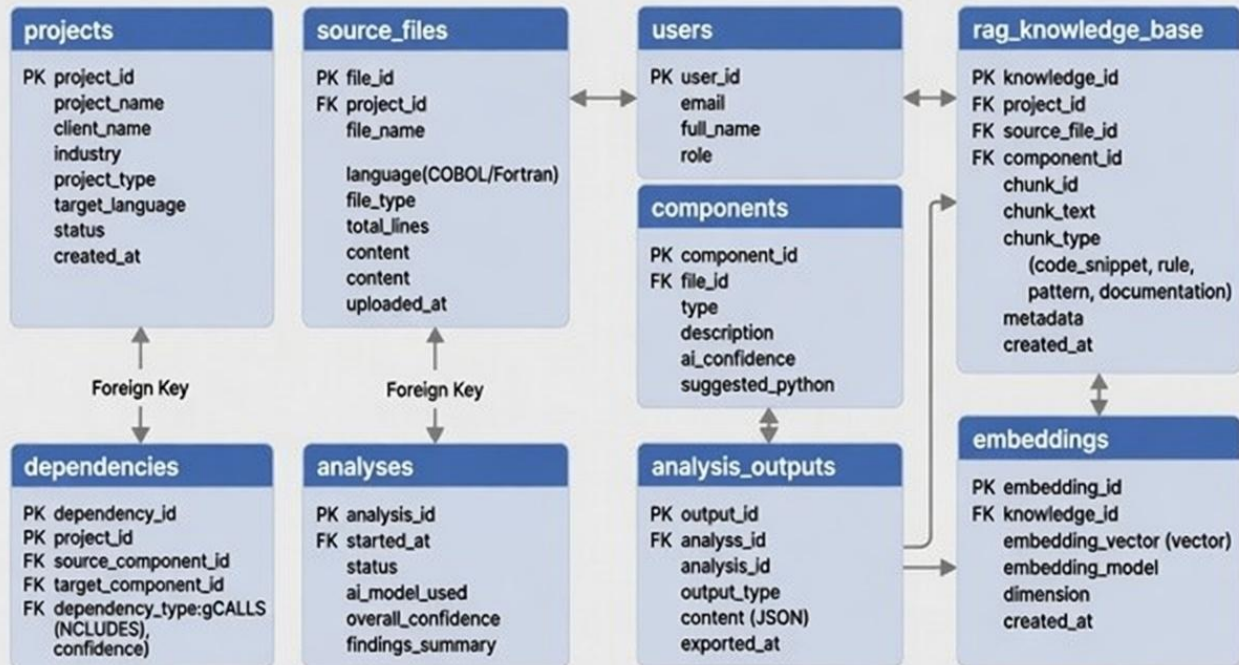
RAG - Why It Matters for Legacy Lift

- Reduce AI errors
 - Prevents the model from guessing or making up logic
 - Uses real code and context
- Preserves Business Logic
 - Legacy systems contain critical rules spread across multiple files
 - RAG ensures the AI sees related code, dependencies, and comments
- Improves Consistency
 - The framework pulls from the same dataset every time as opposed to what is uploaded at each instance.
- Enables Safer Modernization
 - Reduces risk of breaking critical system behavior during conversion

Database Schema

LegacyLift ER Diagram - AI-Assisted Fortran & COBOL Analysis for Python Modernization (with RAG)

AI-Assisted Fortran & COBOL Analysis for Python Modernization with Retrieval Augmented Generation



Project Monetization

Discovery and Assessment

Initial evaluation costs range from \$2,000 to \$10,000, providing tailored project insights for clients.

Framework Implementation

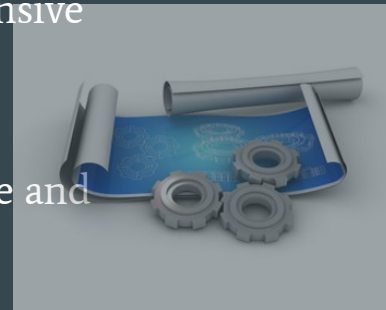
Implementation services are priced \$5,000 to \$20,000, reflecting increased complexity and customization.

Full Migration

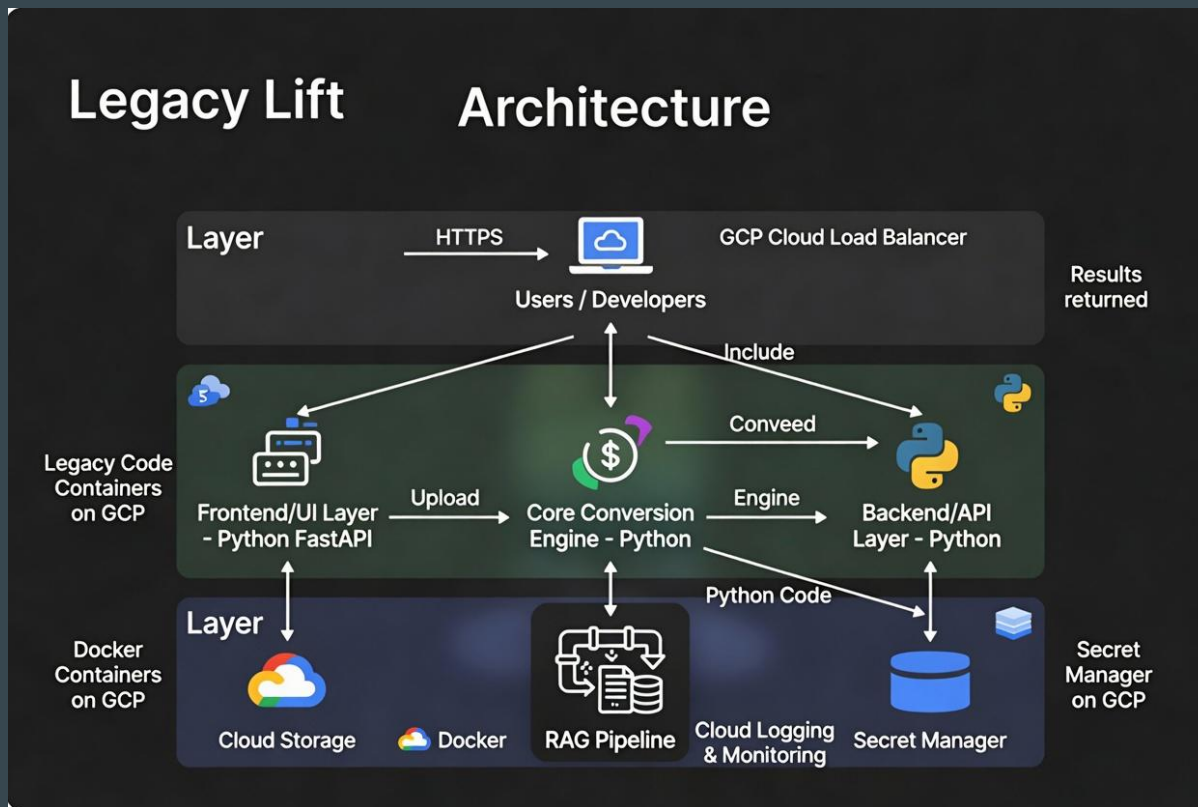
Comprehensive migration services cost \$20,000 to \$80,000, covering extensive technical work and support.

Annual Maintenance

Ongoing maintenance is \$2,000 per year, ensuring continued performance and reliability for clients.



Major Functional Components Diagram



Driver

EXPLORER

COBOL_OOP_MODERNIZATION

.vscode

data

notebooks

01_data_exploration.ipynb

02_pattern_detection.ipynb

03_refactoring_demo.ipynb

output

output_refactored_account.py

src

__pycache__

cobol_parser.py

models.py

refactorer.py

.gitignore

README.md

requirements.txt

run_demo.py

run_demo.py > ...

```
1 # run_demo.py - Quick test for the COBOL to OOP modernization mockup
2 from src.refactorer import OOPRefactorer
3
4 print("🚀 ML-Driven Legacy Code Modernization - Quick Demo\n")
5
6 refactorer = OOPRefactorer()
7
8 # Test on the main example
9 python_code = refactorer.refactor('data/simple_account.cbl')
10
11 print("=== GENERATED PYTHON OOP CODE ===\n")
12 print(python_code)
13
14 # Save with proper UTF-8 encoding
15 with open('output/output_refactored_account.py', 'w', encoding='utf-8') as f:
16     f.write(python_code)
17
18 print("\n✅ Successfully saved to: output/output_refactored_account.py")
19 print("You can now open this file in any editor (VS Code recommended).")
```

PROBLEMS

OUTPUT

TERMINAL

PORTS

QUERY RESULTS

DEBUG CONSOLE

Business logic preserved and modernized from original COBOL

✅ Successfully saved to: output/output_refactored_account.py

You can now open this file in any editor (VS Code recommended).

PS C:\Users\misap\OneDrive\Documents\ODU University\01Spring 2026\CS 522\Course Project\New-Files\cobol_oop_modernization> (C:\ProgramData\miniconda3\shell\condabin\conda-hook.ps1) ; (conda activate cs422)

(cs422) PS C:\Users\misap\OneDrive\Documents\ODU University\01Spring 2026\CS 522\Course Project\New-Files\cobol_oop_modernization>

pwsh

Python Deb...

main

0 0 0

Indexing completed.

Spencer Peloquin (2 days ago) Ln 15, Col 44 Spaces: 4 UTF-8

Implementation

```
src > cobol_parser.py > parse_cobol
1 import re
2 from typing import Dict, List
3
4 def parse_cobol(file_path: str) -> Dict:
5     with open(file_path, 'r', encoding='utf-8', errors='ignore') as f:
6         content = f.read().upper()
7
8         data = {
9             'program_id': re.search(r'PROGRAM-ID\.\s*(\w+)', content),
10            'data_items': re.findall(r'0[1-9]\s+\w+.*PIC\s+(\w+)\s+', content),
11            'paragraphs': re.findall(r'(\w+)-?\w*\.\s*(.*)?(=\w+|\w*\.\s+STOP RUN)', content, re.DOTALL)
12        }
13
14        # Extract potential class fields from DATA DIVISION
15        fields = []
16        for item in data['data_items']:
17            match = re.search(r'(\w+)\s+PIC\s+(.+)', item)
18            if match:
19                fields.append({'name': match.group(1), 'pic': match.group(2)})
20
21        # Paragraphs as potential methods
22        methods = [p[0] for p in data['paragraphs'] if p[0] not in ['MAIN-LOGIC', 'STOP']]
23
24        return {
25            'program_name': data['program_id'].group(1) if data['program_id'] else 'UNKNOWN',
26            'potential_attributes': fields,
27            'potential_methods': methods,
28            'raw_paragraphs': data['paragraphs']
29        }
```

PROBLEMS OUTPUT TERMINAL PORTS QUERY RESULTS DEBUG CONSOLE

Business logic preserved and modernized from original COBOL

✅ Successfully saved to: output/output_refactored_account .py

You can now open this file in any editor (VS Code recommended).

PS C:\Users\misap\OneDrive\Documents\ODU University\01Spring 2026\CS 522\Course Project\New-Files\cobol_oop_modernization> (C:\ProgramData\miniconda3\shell\condabin\conda-hook.ps1) ; (conda activate cs422)

(cs422) PS C:\Users\misap\OneDrive\Documents\ODU University\01Spring 2026\CS 522\Course Project\New-Files\cobol_oop_modernization> n> []

Output

File Edit Selection View Go Run Terminal Help

run_demo.py output_refactored_account.py 01_data_exploration.ipynb 03_refactoring_demo.ipynb

EXPLORER

- COBOL_OOP_MODERNIZATION
 - .vscode
 - data
 - notebooks
 - 01_data_exploration.ipynb
 - 02_pattern_detection.ipynb
 - 03_refactoring_demo.ipynb
 - output
 - output_refactored_account.py
 - src
 - __pycache__
 - cobol_parser.py
 - models.py
 - refactorer.py
 - .gitignore
 - README.md
 - requirements.txt
 - run_demo.py

output > output_refactored_account.py > ...

```
1 from decimal import Decimal
2 from typing import Any
3
4 class AccountClass:
5     """
6     Modern OOP version of COBOL program: ACCOUNT
7     Generated by ML-Driven Legacy Code Modernization framework
8     """
9
10    def __init__(self, account_number: int = 0, owner: str = ''):
11        # Data Division → Class Attributes
12        self.number: int = None # from PIC 9(10)
13        self.balance: Decimal = None # from PIC S9(10)V99
14        self.owner: str = None # from PIC X(50)
15        self.amount: Decimal = None # from PIC S9(8)V99
16        self.number = account_number
17        self.owner = owner
18        self.amount: Decimal = Decimal('0.00')
19        self.balance: Decimal = Decimal('0.00')
20
21    # == Business Logic Methods (Procedural Patterns Grouped via Clustering) ==
22
23    def division(self):
24        """
25        Implements clustered logic from COBOL paragraphs: DIVISION, PROGRAM, ACCOUNT, DIVISION
26        """
27        # From paragraph: DIVISION
28        # TODO: Translate remaining COBOL logic here
29        pass
30
31    # From paragraph: PROGRAM
```

PROBLEMS OUTPUT TERMINAL PORTS QUERY RESULTS DEBUG CONSOLE

Business logic preserved and modernized from original COBOL

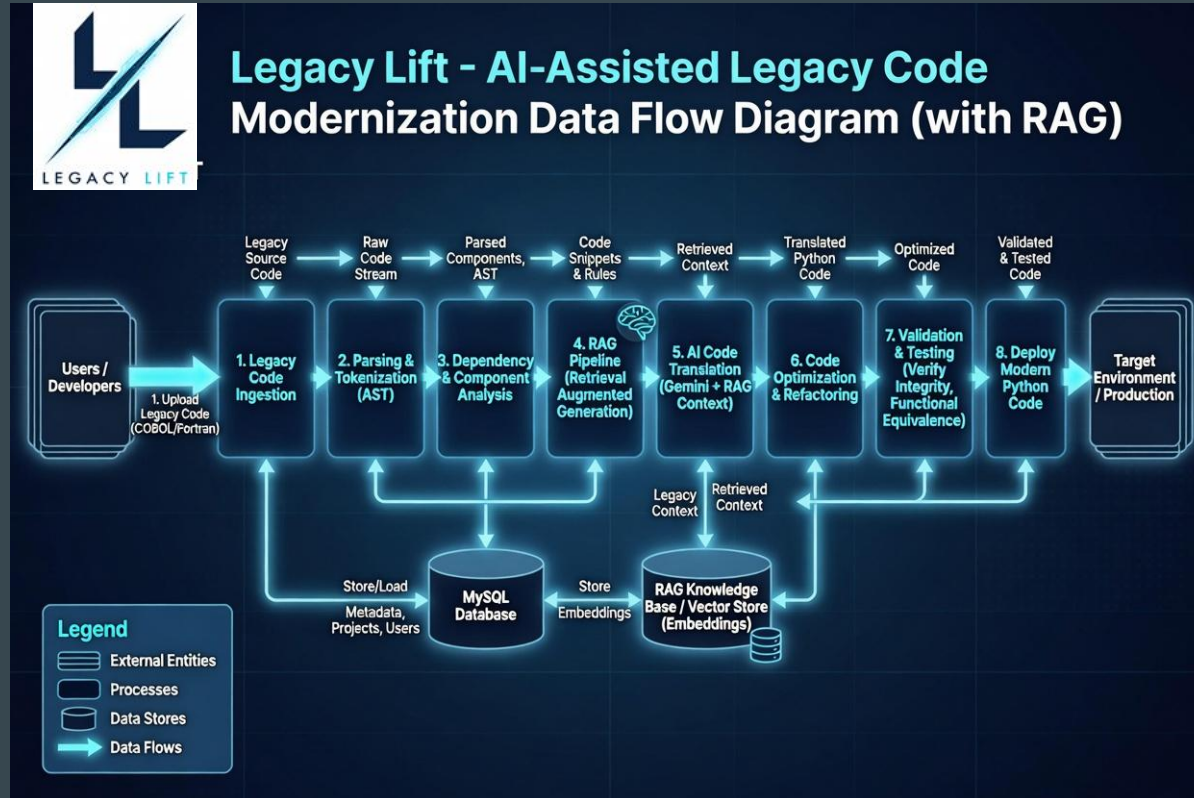
✓ Successfully saved to: output/output_refactored_account .py
You can now open this file in any editor (VS Code recommended).
PS C:\Users\misap\OneDrive\Documents\ODU University\01Spring 2026\CS 522\Course Project\New-Files\cobol_oop_modernization> (C:\ProgramData\miniconda3\shell\conda\bin\conda-ho
k.ps1) ; (conda activate cs422)
• (cs422) PS C:\Users\misap\OneDrive\Documents\ODU University\01Spring 2026\CS 522\Course
Project\New-Files\cobol_oop_modernization>

OUTLINE
TIMELINE
PROJECTS

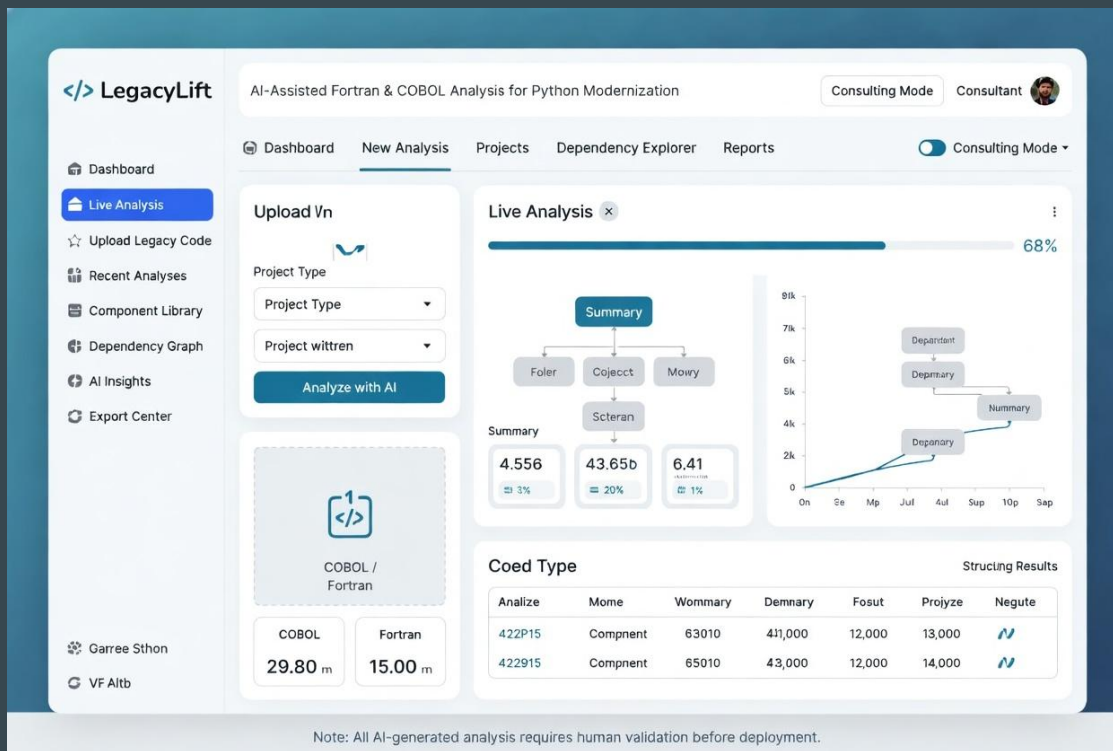
Major Functional Components Diagram



Data Flow Diagram



Project GUI Mockup





Conclusion

Legacy Lift is a viable solution for small businesses seeking a stable, cost effective means for updating and modernizing their legacy code infrastructure.

Questions?



References

“12 Best Legacy Modernization Tools for Companies in 2026.” *12 Best Legacy Modernization Tools for Companies in 2026*, 2 Mar. 2026, launchpad.io/blog/best-legacy-modernization-tools.

“2025 Legacy Code Stats.” *Pragmatic Coders*, 2 Sept. 2025, www.pragmaticcoders.com/resources/legacy-code-stats.

The 2025 State of Legacy Software Modernization Report, www.saritasa.com/wp-content/uploads/2025/07/Saritasa-2025_State_of_Legacy_Software_Modernization_Report.pdf. Accessed 11 Mar. 2026.

“44 Legacy System Modernization Statistics Every Enterprise Should Know in 2026.” *DreamFactory [LIVE]*, www.dreamfactory.com/hub/legacy-system-modernization-statistics. Accessed 11 Mar. 2026.

Ellienosrat, et al. “Best Practices for Leveraging Azure Openai in Code Conversion Scenarios: Microsoft Community Hub.” *TECHCOMMUNITYMICROSOFT.COM*, 26 Mar. 2025, techcommunity.microsoft.com/blog/azure-ai-foundry-blog/best-practices-for-leveraging-azure-openai-in-code-conversion-scenarios/4395993.

“IBM Watsonx Code Assistant for Z.” *IBM*, 12 Apr. 2024, www.ibm.com/products/watsonx-code-assistant-z.

“Lost in Translation: What the AI Code Debate Keeps Getting Wrong.” *IBM Newsroom*, newsroom.ibm.com/blog-lost-in-translation-what-the-ai-code-debate-keeps-getting-wrong. Accessed 11 Mar. 2026.

Sabrina. “Legacy Software Modernization in 2025: Survey of 500+ U.S. It Pros.” *Saritasa*, 25 Aug. 2025, www.saritasa.com/insights/legacy-software-modernization-in-2025-survey-of-500-u-s-it-pros.

“The Dark Side of AI: Assessing Liability When Bots Behave Badly.” @EbgLaw, 22 Sept. 2025, www.ebgLaw.com/insights/publications/the-dark-side-of-ai-assessing-liability-when-bots-behave-badly.



Appendix User Stories - technology consulting firms

As a technology consulting firm, we need to ensure the COBOL to python conversion process allow for a uninterrupted on business operations.

As a technology consulting firm, we wish a silent execution of the conversion process, so it must run parallel to existing systems.

As a technology consulting firm, we wish the converted python code to run similarly in behavior to the original COBOL code, so the workflow isn't interrupted and so there isn't major retraining.

As a technology consulting firm, we wish the implementation to be as short as possible, to reduce exposure to downtime.

As a technology consulting firm, we need the python code to be compatible with the firm's existing GUI and interfaces.

As a technology consulting firm, we need the ability of a rollback function, that gives the company the opportunity to revert any changes made so the client can be further revised.

As a technology consulting firm, we need the converted python 3.11 to be immediately available and production ready.

As a technology consulting firm, we need to guarantee strong data protection and security controls, so customer data remains protected and uncorrupted.



Appendix User Stories - Production Engineer

As a production engineer, I wish the Legacy Lift Python code to have identical functionality to the original COBOL code in system tests.

As a production engineer, I wish the legacy Lift to log any changes made during translation process.

As a production engineer, I need the finished product to be able to run on modern hardware.

As a cybersecurity admin, I wish the finished product to have no new zero day vulnerabilities.

As a cybersecurity admin, I wish the finished product to fix all security flaws known and unpatched in the original product.

As a production engineer, I wish the finished product to use fewer resources than the original product.

As a production engineer, I wish the finished product to have as few redundancies in production as possible to improve performance.

As a production engineer, I need the framework to have enough tokens purchased so that it does not stall mid-conversion.

As a production engineer, I wish there to be an API to embed existing frameworks into LegacyLift.



Appendix User Stories - Enterprise

As a banking firm, I wish to make sure that I do not need to rip out and reinstall all of my ATMs just to accommodate the new system.

As an enterprise, I need to make sure that there is some sort of fallback option in case the conversion fails catastrophically.

As a potential enterprise client, I need to make sure that the framework is completely scalable, from a sole accountant's work to a local or regional bank.

As an enterprise, I wish the finished product to be compatible with the existing enterprise toolchain so all plugins are compatible.

As an enterprise, I need the finished product to be fully compatible with the existing SQL workflows.

As an enterprise, I need the finished product to integrate with the company's website and backend to avoid a larger conversion.

As an enterprise, I need the conversion to occur in a sandbox to avoid disrupting existing systems.

As an enterprise, I wish the AI code to be sanitized to avoid corruption or copyright violations.

As an enterprise, I wish existing filesystem hooks to be preserved in order to maintain compatibility.

As an enterprise, I do not wish the finished product to sanitize the existing codebase just to make it look prettier, as poorly written code enables functionality that could break if altered.

As an enterprise, I need all of the data sanitized so that no PII is leaked.